Target-specific sparse correction certificates recover stochastic language trajectories across numerical precision

Lei Ke

Nature-style working paper; not a journal-formatted submission.

Stochastic language generation is difficult to reproduce across numerical precision because one changed token can propagate through an autoregressive trajectory. Here we introduce sparse precision replay certificates (SPRCs), target-specific correction records built from a public integer next-token contract. Exact corrected replay is a protocol invariant; the empirical questions are how often corrections occur and what they cost under a declared package boundary. In Qwen2.5-1.5B-Instruct experiments spanning 20 English and Chinese prompts and three public seeds, uncorrected FP16-to-BF16 replay reproduced 10 of 60 trajectories, whereas complete manifests recovered 60 of 60. Corrections affected 2.16% of tokens on average (prompt-cluster bootstrap 95% confidence interval, 1.80-2.53%), and 58 of 60 outputs reached a public sentence endpoint. Reversing precision retained exact recovery in 20 of 20 additional trials. On the frozen 20-prompt seed-0 bundle, a compact referenced SPRC used 1,148 bytes, 24.76% of the matching 4,636-byte full trace and less than fixed block-repair baselines. Removing the logit-bin and integer-mass contracts at the same top-two support increased correction density by 1.123 percentage points (paired prompt bootstrap 95% interval, 0.171-2.015). A distribution-free audit bounded the top-two, 16-bit integer-apportionment total variation below 3.052×10^{-5} per step. These results establish compact target-specific replay for one known model and GPU stack. They do not establish target-independent determinism, native-distribution preservation, semantic equivalence or cross-hardware generality.

Keywords: stochastic inference, numerical precision, exact replay, language models, reproducibility, probability contracts

Reproducible inference is a prerequisite for comparing language models, auditing generated outputs and reconstructing computational experiments. This requirement is unusually difficult for stochastic autoregressive generation. Even when model weights, prompts, decoding parameters and pseudo-random seeds are held fixed, a small numerical change in one next-token distribution can move a sample across a decision boundary. The changed token then alters every subsequent conditional distribution. Recent work has documented substantial response and evaluation variability caused by batch configuration, accelerator type and reduced-precision arithmetic, including

divergent reasoning paths and response lengths under nominally identical evaluation settings [1]. The practical question is therefore not only whether an implementation is deterministic on one machine, but whether a stochastic trajectory can be reconstructed when the numerical environment changes.

Existing approaches address complementary parts of this problem. Higher-precision execution can reduce numerical drift but increases memory or computational cost [1]. Verification methods can test whether an output remains plausible under a reference implementation, but verification does not itself reconstruct a chosen stochastic path [2]. Shared-randomness sampling methods developed for generative communication, including sparse sampling and range-coding approaches, demonstrate that public probability partitions can couple a message or random stream to autoregressive generation [3,4]. Their implementation, however, depends on candidate support, ordering and finite-precision probability boundaries. Finite-precision artifacts have also been shown to create detectable low-probability effects in language-model sampling pipelines [5]. Together, these findings motivate an explicit contract between probability computation and stochastic replay.

A strict bitwise contract over all model operations would be expensive and brittle. Conversely, recording the complete generated token sequence always guarantees replay but provides no compression and reveals nothing about where numerical precision actually changes a decision. We study an intermediate question: can a public discrete next-token contract make cross-precision disagreements sufficiently sparse that exact replay requires only a small correction record? This question separates two properties that are often conflated. Exact recovery is a deterministic property of applying a complete correction manifest. Sparsity is an empirical property of how often two numerical environments disagree under a specified contract.

We introduce sparse precision replay certificates (SPRCs) for target-specific stochastic reconstruction. At each generation step, relative logits are quantized to public integer bins, candidates are ranked under a fixed top-k envelope and contract candidates are ordered by token identifier. Their quantized weights are apportioned into an integer mass of size 2^B, and a public pseudo-random value selects a token. To construct a certificate, the reference trajectory is evaluated under the intended target environment while conditioning on the reference prefix. The certificate stores only pairs (t, x_t) for steps at which the target contract choice differs from reference token x_t. During replay, corrections are applied before extending the prefix, which prevents a local disagreement from cascading. This is a target-conditioned sparse delta representation; the scientific question is whether the public probability contract makes that delta sparse and auditable enough to improve on coarser repair records under the same assumptions.

We first establish compatibility with the published SparSamp artifact on GPT-2 and then

evaluate the replay design on the instruction-tuned Qwen2.5-1.5B model [3,6]. The principal experiment contains 20 bilingual prompts, three public seeds and 60 FP16-to-BF16 trajectories. We measure correction density as the primary empirical endpoint, with exact replay as an integrity gate, and separately report package bytes, target passes, sentence completion, candidate coverage and distributional costs. Matched seed-only, full-trace and block-repair controls use the same referenced-package boundary. The resulting claim is deliberately bounded to a known model, tokenizer, prompt, sampling configuration and target numerical environment.

Results

Official SparSamp compatibility reproduction establishes artifact fidelity. Before modifying the probability contract, we ran the unchanged SparSamp algorithm from Zenodo record 15025436 on the local GPT-2 checkpoint. The artifact Basic Test generated 105 tokens, embedded 576 bits (5.486 bits per token) and decoded the complete message. We then evaluated the 12 unique configurations needed to reproduce published Tables 2-4: ten message block lengths from 2 to 1023 at top-p=1.0 and block length 64 at top-p=0.8, 0.95 and 1.0. Each configuration used the same deterministic sample of 100 IMDB contexts, yielding 1,200 configured trials.

The compatibility matrix passed both prespecified core gates (Supplementary Table 1 and Supplementary Fig. 1). All 846 completed trials without Token Ambiguity decoded exactly. All 16 capacity comparisons were within 5% relative error of the published embedding-rate or entropy-utilization value. The maximum error was 4.12%. Of 1,200 configured trials, 1,193 completed and seven did not finish a message block within the artifact's 200-token ceiling. These seven occurred only at block lengths 512 and 1023 and were retained as budget-exhausted trials rather than counted as decoding errors or omitted. Token Ambiguity occurred in 347 completed trials and was reported separately, matching the paper's no-ambiguity decoding condition.

The official-reproduction status is `PASS_WITH_LIMITATIONS`. This result is a compatibility reproduction, not a strict dependency recreation. The unchanged algorithm source was executed with PyTorch 2.7.1+cu126 and Transformers 4.41.2 on an RTX 3060 Laptop GPU. Transformers 4.57.6 failed at the artifact's legacy cache interface, and a strict Torch 2.2.2 CUDA environment could not be completed because the upstream wheel transfer repeatedly terminated. Throughput therefore remains hardware- and stack-specific. The capacity and decoding conclusions, rather than absolute speed, define the reproduction result.

Sparse certificates separate exact replay from numerical agreement. To test whether precision-induced disagreements are sparse, we generated a reference sequence in one numerical precision and recomputed the public contract in a second precision while forcing the second run to follow the reference prefix (Fig. 1). The local target choice was compared with the reference token at every step. A correction was emitted only for a mismatch. A fresh target run then used the same public seed and contract, applying the stored correction before appending each token.

The replay invariant is exact by construction. If the replay prefix equals the reference prefix through step $t-1$, both certificate construction and replay evaluate the target contract on the same prefix. When the target choice equals the reference token, no correction is required. Otherwise, the manifest replaces the target choice with the recorded reference token. Induction therefore gives equality of the full token sequence, provided that the target model, tokenizer, prompt, contract configuration and correction manifest are unchanged. This statement does not imply that the reference and target distributions are identical. The empirical question is the number of corrections required to maintain the invariant.

In the GPT-2 pilot, six 64-token seeded trajectories were generated in FP32 and replayed in FP16. Uncorrected replay recovered two of six trajectories, while corrected replay recovered all six. The mean correction rate was 2.34%, with a maximum of 6.25%, and the correction record occupied 3.52% of the corresponding full token trace. Three independent executions produced the same deterministic result signature. This pilot established that the audit pipeline could distinguish timing variability from token-level reproducibility before moving to an instruction-tuned model.

The first Qwen pilot used FP16 references and BF16 replay for six seeded 64-token trajectories. Only one uncorrected trajectory matched exactly, whereas all six corrected trajectories matched. The mean correction rate was 1.56%. The full integer contract agreed at 83.07% of shared-prefix steps, showing that contract differences were more common than final token-choice differences. None of the fixed 64-token outputs ended at a sentence boundary, so exact token reconstruction alone was insufficient as a user-facing completion criterion.

Exact replay scales across bilingual prompts. We next evaluated the seeded policy on 20 prompts covering explanation, comparison, planning, summarization-style responses and structured lists in English and Chinese. Three public seeds were used for each prompt, giving 60 FP16 reference and BF16 replay trajectories. Generation continued for at least 64 tokens and stopped at the first public sentence endpoint, with a maximum of 96 tokens. All configured trials were retained in the analysis.

Certificate-corrected replay recovered 60 of 60 token trajectories, whereas uncorrected replay recovered 10 of 60 (Table 1 and Fig. 2a). All 20 prompt clusters achieved three of three corrected replays. The Wilson 95% interval across the 20 prompt clusters was 0.839-1.000. This interval describes the finite prompt set and does not establish performance on a wider prompt population. The mean correction rate was 2.16% (prompt-cluster bootstrap 95% confidence interval, 1.80-2.53%), with a maximum trial rate of 6.15% and prompt-level variation shown in Fig. 2b. Under the original fixed-width payload-only accounting, the mean correction-record size was 2.88% of a full token trace (95% confidence interval, 2.40-3.36%). This legacy estimate excludes identity and configuration headers. Package-level costs are reported separately below.

The result was consistent across the two language strata. English prompts reached 30 of 30 corrected replays with a mean correction rate of 1.94% (95% confidence interval, 1.47-2.42%). Chinese prompts reached 30 of 30 with a mean rate of 2.39% (1.86-2.86%). These intervals overlap and were not used for a language-difference claim. The mean trajectory length was 75.0 tokens (cluster interval, 72.3-78.1). Without corrections, the common exact prefix averaged 39.45 tokens, illustrating how a small number of local disagreements can produce full-sequence failure.

Every reference token appeared in the target top-eight envelope across 4,500 generated positions. The top-four envelope missed one position, and the largest observed target rank of a reference token was six. This observation supports the use of a bounded validation envelope for the tested configuration, but it is not a guarantee for other models or prompts.

Replay-package cost depends on the audit boundary. R049 measured serialization on the 20-trial seed-0 reference bundle (1,500 generated tokens and 30 corrections). The versioned binary manifest payload was 247 bytes against a 3,715-byte variable-length full-token trace, a payload-only ratio of 6.65%. When a self-contained JSON package included the shared contract header and per-trial identities, the certificate package was 6,136 bytes versus 9,604 bytes for the corresponding full-trace package (63.89%). A compact binary package that referenced the externally shared bundle, model fingerprint and target-environment fingerprint used a fixed 101-byte header and totalled 1,148 bytes versus 4,636 bytes for the referenced full trace (24.76%, or 6.123 bits per generated token). These are different estimands: the first excludes all headers, the second includes all audit metadata, and the third assumes the shared bundle has already been transferred. Certificate construction and corrected replay each required one target-model pass. Measured source-machine throughputs were 11.53 and 11.50 tokens per second, respectively, versus 11.90 tokens per second for the uncorrected evaluation pass. The timing is descriptive for one machine, not a portability claim.

Table 1: Cross-precision replay results for Qwen2.5-1.5B-Instruct. Continuous intervals use 10,000 prompt-cluster bootstrap resamples. Exact recovery counts retain all trials.

Setting	Trials	Corrected exact	Uncorrected exact	Mean correction rate	Legacy fixed-width payload ratio	Sentence complete
FP16 to BF16, top-2, main scale	60	60/60	10/60	2.16% [1.80, 2.53]	2.88% legacy fixed-width [2.40, 3.36]	58/60
FP16 to BF16, top-2, ablation subset	20	20/20	5/20	2.05% [1.24, 2.93]	2.74% [1.67, 3.87]	20/20
FP16 to BF16, top-4	20	20/20	1/20	2.41% [1.95, 2.86]	3.21% [2.60, 3.82]	16/20
BF16 to FP16, top-2	20	20/20	5/20	2.06% [1.33, 2.86]	2.75% [1.74, 3.82]	19/20

Matched replay baselines isolate sparse-record cost. To test whether token-level corrections improve on simpler replay records, we re-analysed the frozen 20-prompt seed-0 bundle using one referenced header and the same per-trial identity records for every method (Table 2). Seed-only replay added no trajectory payload but recovered 5 of 20 paths; its 901 bytes consist entirely of shared and per-trial identity metadata. The SPRC recovered 20 of 20 using 1,148 bytes (6.123 bits per token), compared with 4,636 bytes (24.725 bits per token) for a target-independent full token trace. Fixed block repair stored every reference token in a block containing at least one SPRC correction. Its smallest four-token variant required 1,408 bytes, with larger blocks increasing to 2,553 bytes at block size 32. Thus the sparse record was smaller than each exact block-repair control under the matched boundary.

We next removed the logit-bin and integer-mass contracts while retaining the same HMAC fraction, frozen reference prefix, BF16 target and top-two support. This unquantized top-two delta required corrections at 3.178% of positions, 1.123 percentage points more than SPRC (paired prompt bootstrap 95% interval, +0.171 to +2.015), and used 1,200 referenced bytes. Expanding the unquantized support to a positive-probability top-16 cap increased correction density to 25.454% and package size to 2,371 bytes. One of 1,500 steps retained only five positive-probability candidates in BF16, so this variant is explicitly a positive-support cap rather than an ideal real-arithmetic top-16 distribution. No reference token fell outside the available positive support.

The byte advantage does not imply a compute advantage over direct storage. Full-trace reconstruction requires no target-model pass. SPRC, unquantized delta and block-repair construction evaluate the target on every reference prefix, and operational replay evaluates the target again. A specialized block implementation could skip generation within stored blocks, but that optimization was not timed.

Table 2: Matched referenced-package replay controls on the 20-prompt seed-0 bundle. All byte counts include the same 101-byte referenced header and per-trial identity records.

Method	Exact recovery	Referenced bytes	Ratio to full trace	Bits/token	Target passes	Target-specific
Seed only	5/20	901	19.43%	4.805	1	No
SPRC	20/20	1,148	24.76%	6.123	2	Yes
Unquantized top-two delta	20/20	1,200	25.88%	6.400	2	Yes
Full token trace	20/20	4,636	100.00%	24.725	0	No
Four-token block repair	20/20	1,408	30.37%	7.509	2	Yes
Eight-token block repair	20/20	1,655	35.70%	8.827	2	Yes
Sixteen-token block repair	20/20	1,963	42.34%	10.469	2	Yes
Thirty-two-token block repair	20/20	2,553	55.07%	13.616	2	Yes
Unquantized top-16-cap delta	20/20	2,371	51.14%	12.645	2	Yes

A public sentence rule improves structural completion. Fixed token budgets can produce exact but visibly truncated outputs. We therefore separated token replay from structural completion. In the sentence-completion pilot, generation continued beyond token 64 until a public terminal-punctuation rule was satisfied or token 96 was reached. For six seeded trajectories, sentence completion improved from zero of six under the fixed 64-token setting to six of six under the public stopping rule, at an average cost of 9.5 additional tokens. Exact corrected replay remained six of six and the mean correction rate was 1.62%.

At scale, 58 of 60 reference trajectories satisfied the structural endpoint. Failures remained in the denominator and were not removed from replay analysis. This endpoint establishes only that the generated text ended at recognized sentence punctuation. It does not measure factuality, coherence or human preference. Blinded semantic evaluation remains required before making a text-quality claim.

Contract width exposes a distribution-reliability Pareto frontier. Restricting the contract to the two highest-probability candidates can make decisions stable but conditions generation on a smaller fraction of the model distribution. To quantify this trade-off, we compared top-two and top-four contracts on the same 20 prompts and public seed. Because contract width participates in token selection, the two settings intentionally generated different reference trajectories. The comparison is paired by prompt and seed, not by token sequence.

Expanding the contract from two to four candidates increased retained source mass from 0.734 to 0.845, a paired increase of 0.110 (95% confidence interval, 0.090-0.131, Fig. 3a). The measured truncation component decreased from 0.382 to 0.198 nats per token, a change of -0.184 (-0.220 to -0.150, Fig. 3b). The full-logit quantization term remained much smaller: 0.00257 nats per token for top-two and 0.00247 for top-four in these runs. These components are reported separately. Their numerical sum is not asserted to equal total divergence after integer apportionment.

The distributional improvement did not produce a reliability improvement. Correction density changed by +0.35 percentage points (-0.65 to +1.34, Fig. 3c), so the interval included no change. Shared-contract exactness fell by 25.14 percentage points (-27.70 to -22.72, Fig. 3d). Sentence completion decreased from 20 of 20 to 16 of 20, a paired change of -20 percentage points (-40 to -5). Because completion is an automated structural endpoint and this ablation used one seed, the decrease is treated as a bounded signal rather than a general semantic-quality result. We therefore retain top-two as the default reproducibility operating point and top-four as a lower-truncation alternative on the Pareto frontier.

Bounded parameter sensitivity identifies integer-mass saturation. We next froze a one-factor-at-a-time sensitivity contract around the seed-0 top-two baseline. Six new 20-prompt variants changed logit quantum to 0.25 or 1.0, temperature to 1.0 or 1.4, or integer mass bits to 12 or 20; the existing top-four experiment supplied the contract-width comparison. All 120 new trajectories and the 20 baseline trajectories passed the corrected-exact integrity gate (Table 3). Correction density below is the aggregate correction count divided by aggregate generated tokens. Prompt-paired bootstrap intervals account for the fixed bilingual prompt set but not for model, seed or hardware variation.

The lowest observed correction density was 1.721% at 20 mass bits, compared with 2.000% at the 16-bit baseline. The paired change was -0.279 percentage points (95% interval, -1.159 to +0.537) and therefore did not exclude no change. Its referenced package used 1,131 bytes versus 1,148 baseline bytes, with the difference reflecting both manifest contents and changed trajectory lengths. Reducing integer mass to 12 bits increased correction density to 3.074%; lowering temperature to 1.0 increased retained source mass from 0.734 to 0.816 and increased corrections to 3.070%. None of the six paired correction-rate intervals excluded zero. These results rule out a monotonic "more precision is always better" interpretation and identify 20 mass bits as a replication candidate, not a confirmed optimum.

Table 3: Prespecified one-factor-at-a-time parameter sensitivity. Each row contains the same 20 prompts and public seed 0. Delta intervals use 10,000 prompt-paired bootstrap resamples. Referenced bytes include the common 101-byte header and per-trial identity records.

Variant	**q**	**T**	**B**	**Exact recovery**	**Correction density**	**Delta vs baseline, pp (95% CI)**	**Retained mass**	**Referenced bytes**
Baseline	0.5	1.2	16	20/20	2.000%	reference	0.734	1,148
q=0.25	0.25	1.2	16	20/20	2.498%	+0.498 (-0.611 to +1.546)	0.747	1,178
q=1.0	1.0	1.2	16	20/20	2.056%	+0.056 (-0.899 to +1.006)	0.752	1,141
T=1.0	0.5	1.0	16	20/20	3.070%	+1.070 (-0.253 to +2.364)	0.816	1,212
T=1.4	0.5	1.4	16	20/20	2.364%	+0.364 (-0.798 to +1.429)	0.678	1,168
B=12	0.5	1.2	12	20/20	3.074%	+1.074 (-0.401 to +2.507)	0.752	1,212
B=20	0.5	1.2	20	20/20	1.721%	-0.279 (-1.159 to +0.537)	0.747	1,131

Replay is stable in both FP16/BF16 directions within the tested environment. To test whether the result depended on selecting FP16 as the reference, we reversed the direction for the 20-prompt ablation (Fig. 4). BF16 references replayed in FP16 achieved 20 of 20 corrected trajectories, with a correction rate of 2.06% (1.33-2.86%) and 19 of 20 sentence-complete outputs. Relative to the matched FP16-to-BF16 top-two subset, the correction-rate change was +0.01 percentage points (-1.00 to +1.00), and the retained-mass change was -0.0043 (-0.0193 to +0.0104). Neither interval excluded zero.

These data support bidirectional precision replay on the tested model and GPU stack. They do not show equivalence of FP16 and BF16 distributions. Only five of 20 reference trajectories were identical across the two reference precisions, and the quantization total-variation term was slightly larger in the reverse experiment. The certificate recovers a selected reference path despite distributional differences. It does not remove those differences.

Discussion

Our results show that target-specific cross-precision stochastic language trajectories can be reconstructed with a compact correction record when the reference and intended target environments are known. A complete manifest guarantees replay by construction; it is not an empirical accuracy claim. The empirical result is that disagreements remained sparse across 4,500 Qwen token positions, 20 bilingual prompts and three public seeds. On the seed-0 bundle, token-level SPRCs were smaller than matched block-repair records, but a full trace remained computationally simpler and target-independent.

SPRCs complement approaches that suppress or detect nondeterminism. Higher-precision computation aims to prevent numerical divergence [1], whereas SPRCs permit a specified target environment to differ and record only the decisions needed to reconstruct a selected path. Token-level verification methods determine whether outputs are consistent with a trusted reference [2]. The present method instead emits enough information to recover a particular trajectory. Shared-random sampling methods such as SparSamp and range coding motivate deterministic probability partitions [3,4], while finite-precision detection results show why implementation-level probability contracts cannot be treated as ideal real arithmetic [5]. Our experiments operationalize this distinction by reporting full-logit quantization, support truncation, contract agreement and correction density separately.

The top-k ablation identifies a practical design rule. A wider support retains more probability

mass and lowers the truncation component, but creates more opportunities for cross-precision candidate and integer-boundary disagreement. In the present sample, top-four therefore improves one distributional objective while worsening contract agreement and structural completion. This is a Pareto trade-off rather than a uniformly superior configuration. The matched unquantized top-two result isolates a small but measurable probability-contract contribution: removing bins and integer mass increased correction density by 1.123 percentage points and referenced size by 52 bytes. In contrast, widening the unquantized support to 16 produced a much larger correction increase. The integer-apportionment audit shows that the 16-bit allocation term is bounded far below the observed quantization TV at top-two. R053 nevertheless found that reducing mass resolution to 12 bits raised the observed correction density, whereas 20 bits produced the lowest point estimate but an interval that included no change. Integer resolution therefore has a practical floor and a possible saturation region; the current one-seed evidence does not identify a universal optimum. Related work on list decoding, dyadic approximation and tokenization synchronization addresses capacity or text-channel synchronization under different objectives [7-9]. Those mechanisms are potential composition layers, not evidence for target-independent replay.

Several limitations define the current result. First, certificates are target-specific: construction evaluates the target precision on the reference prefix, so a certificate for BF16 is not guaranteed to work for a different accelerator, kernel, model revision or tokenizer. Second, all Qwen experiments used one RTX 3060 Laptop GPU and one software stack. Independent hardware replication is therefore required for a cross-system claim. Third, the top-two contract retains approximately three quarters of the quantized source mass on average and incurs a substantial truncation component. Exact replay must not be interpreted as distribution preservation. Integer apportionment adds a further finite-mass approximation that was not isolated as a separate divergence term in the main artifacts. Fourth, public sentence completion is not a semantic or factual-quality measure. Fifth, this study reconstructs token identifiers under a shared tokenizer and does not test recovery after public-text re-tokenization, normalization or rewriting.

Within these boundaries, sparse certificates provide a research instrument for auditing where precision changes a sampled token. Their best current use case is a frozen benchmark or incident record for a known recipient environment, not arbitrary portability. The external bundle and checkpointed target runner make an independent replay test operational, but the current 20-trial result is a same-machine smoke test. Native, top-two and top-four materials have been prepared for a blinded quality study, but no human ratings were collected. Independent hardware replication, broader parameter sensitivity and an approved blinded comparison should precede claims of hardware generality, globally optimal contract design or semantic equivalence.

Methods

Models and software environment. Experiments used a local Qwen2.5-1.5B-Instruct checkpoint [6] and a local GPT-2 checkpoint. Qwen inference ran on an NVIDIA GeForce RTX 3060 Laptop GPU with 6 GB memory. The recorded Qwen environment used Python 3.11.15, PyTorch 2.7.1 with CUDA 12.6 and Transformers 4.57.6. The isolated official-artifact compatibility environment used the same Python, PyTorch and CUDA versions with Transformers 4.41.2. Model weights, tokenizers and experiment outputs were loaded locally. EOS and padding tokens were excluded from replay-certificate sampling so that the public sentence rule, rather than model EOS, determined the evaluated stopping point.

Official artifact compatibility protocol. The official SparSamp archive was obtained from Zenodo record 15025436 and its archive MD5 was verified before extraction. The experiment runner imported the artifact's `encode_spar` and `decode_spar` functions without modifying their algorithm code. Source SHA-256 hashes, the local GPT-2 directory fingerprint, software versions and trial configuration were stored in the checkpoint. The paper reports 100 IMDB contexts, whereas the artifact `get_statistics.py` comment states 100 but samples 10 when more than 100 are available. We followed the paper-level sample size and selected 100 contexts once with public seed 42. Each trial used artifact seed 777, a minimum of 100 and maximum of 200 generated tokens, and the artifact message starting at offset 19.

The matrix combined duplicate requirements into 12 unique configurations. Table 2 was represented by block lengths 2, 4, 8, 16, 32, 64, 128, 256, 512 and 1023 at top-p=1.0. Tables 3-4 were represented by block length 64 and top-p values 0.8, 0.95 and 1.0. Trials were written atomically after each context. A configuration mismatch prevented checkpoint merging. Re-encoding the rendered public text was used to identify Token Ambiguity. Completed trials with ambiguity were excluded only from the paper's no-ambiguity decoding and capacity denominators and remained counted in the trial ledger. Failure to complete a full message block within 200 tokens was recorded as budget exhaustion, following the artifact's own distinction between this event and an embedding error.

For each configuration, embedding rate was the ratio of total encoded bits to total generated tokens across eligible contexts. Entropy utilization was total encoded bits divided by total model entropy. The reproduction gate required exact decoding in every completed no-ambiguity trial and at most 5% relative error for the published Table 2 utilization values and Table 4 SparSamp embedding-rate and utilization values. Two-sided 95% percentile intervals used 10,000 context bootstrap resamples with public seed 20260721. Sampling time and throughput were reported descriptively because the reproduction hardware differed from the paper's hardware.

Prompt and trajectory design. The main Qwen experiment contained 20 fixed prompts: ten English and ten Chinese prompts. They covered explanations, comparisons, practical plans and structured responses in general educational and software topics. Each prompt was evaluated with public seeds 0, 1 and 2 under the seeded policy, producing 60 trajectories. Seeds are repeated stochastic trajectories within a prompt and were not treated as independent prompt samples for continuous-metric intervals.

Reference generation used FP16 and target replay used BF16 in the main experiment. Generation continued for at least 64 tokens and stopped at the first output satisfying the public sentence-completion predicate, with a hard maximum of 96 tokens. R046 reversed the reference and replay precision on all 20 prompts with seed 0. The top-*k* ablation compared contract widths two and four on the same prompts and seed. No completed trial was excluded.

Quantized next-token snapshot. At generation step t, the model produced logits $\ell_{t,i}$. After blocking EOS-like tokens, logits were shifted by their finite maximum and quantized to a public grid of width $q = 0.5$:

$$b_{t,i} = \left\lfloor \frac{\ell_{t,i} - \max_j \ell_{t,j}}{q} + \frac{1}{2} \right\rfloor .$$

The quantized logits $qb_{t,i}$ were divided by temperature $T = 1.2$ and normalized. Candidates were ranked with stable token-identifier tie breaking. A top-16 envelope was retained for validation. The contract then used the highest-ranked $k \in \{2, 4\}$ candidates and ordered them by integer token identifier before mass allocation. Candidate rank always refers to probability rank before canonical reordering.

We measured the Kullback-Leibler divergence from the unquantized temperature-scaled distribution to the full quantized-logit distribution and their total variation. These are logit-quantization terms. Let A_t denote retained contract support and Z_t its probability mass under the quantized full distribution. Before integer apportionment, conditioning on A_t gives

$$D_{\mathrm{KL}}(Q_t \,\|\, \widetilde{P}_t) = -\log Z_t.$$

This is the reverse-KL truncation component. It is not $D_{\mathrm{KL}}(\widetilde{P}_t \,\|\, Q_t)$, which is infinite when positive source support is removed. We do not claim that the truncation and full-logit quantization numbers sum to total divergence after integer apportionment.

Integer probability contract. For contract token identifiers $v_1, \ldots, v_k$ and quantized bins $b_1, \ldots, b_k$, Decimal arithmetic at precision 80 computed weights

$$w_i = \exp\left(\frac{q(b_i - \max_j b_j)}{T}\right).$$

The contract used a total integer mass $M = 2^{16}$. One count was reserved for each retained token. The remaining $M - k$ counts were assigned proportionally to w_i by deterministic largest-remainder apportionment. Remainder ties were resolved by token identifier. The resulting counts are non-negative, preserve retained support and sum exactly to M.

Let p_i be the normalized retained-support weight before apportionment, a_i the largest-remainder allocation of $(M - k)p_i$, $c_i = 1 + a_i$ the implemented count and $r_i = c_i/M$. If $e_i = a_i - (M - k)p_i$, then

$$r_i - p_i = \frac{1 - kp_i + e_i}{M}, \qquad -1 < e_i < 1.$$

Every positive coordinate error is therefore strictly below $2/M$. Because coordinate errors sum to zero, at most $k - 1$ can be positive, giving the distribution-free bound

$$\mathrm{TV}(p, r) < \frac{2(k-1)}{M}.$$

For the top-two, 16-bit contract this is $1/32768 = 3.0518 \times 10^{-5}$ per step. We verified the applicable k and M on all 1,500 saved seed-0 contracts. This term is separate from full-logit quantization and support truncation. No finite distribution-free KL bound exists without a public positive lower bound on every p_i, so we do not infer one from the TV bound.

For public seeded sampling, a key was derived by hashing the public seed. HMAC-SHA256 over the model name, prompt, contract parameters, seed and step produced a rational sample that was mapped to one of the integer cumulative intervals. Greedy controls selected the largest count with token identifier as a tie breaker. The public-seed construction supports reproducible experiments. It is not a cryptographic secrecy claim.

Sparse precision replay certificate. Let $x_{1:n}$ be the reference token trajectory and let $g_t(x_{<t})$ be the token selected by the target-precision contract at step t when conditioned on the reference prefix. The manifest is

$$C = \{(t, x_t) : g_t(x_{<t}) \neq x_t\}.$$

During replay, the target environment computes its local contract token on the reconstructed prefix and applies

$$y_t = \begin{cases} x_t, & (t, x_t) \in C, \\ g_t(y_{<t}), & \text{otherwise.} \end{cases}$$

If the model, tokenizer, prompt, configuration, public seed and target numerical environment match certificate construction, induction on t yields $y_{1:n} = x_{1:n}$. This is a conditional safety property, not a termination or target-independence theorem.

The operational procedure is:

1. The constructor generates and freezes reference tokens, then evaluates the intended target contract on each reference prefix.
2. A correction pair is emitted exactly when the target choice differs from the frozen reference token.
3. The recipient verifies bundle, model and target-environment identities before replay.
4. Replay recomputes the target choice on the reconstructed prefix and substitutes a stored token at correction steps.
5. An auditor checks token-sequence equality and package identities; exact equality is an integrity gate, while correction density and bytes are measured outcomes.

For a trajectory of length n, construction performs one target-model pass conditioned on the frozen reference prefixes and stores at most one correction pair per step. Replay performs one target-model pass and one manifest lookup per step. With a hash-indexed correction set, replay is $O(n)$ in target evaluations and $O(c)$ manifest storage for c corrections; the referenced binary serialization is $O(c)$ apart from the fixed shared header. These costs describe the implemented protocol and do not imply target-independent generation.

The compact referenced package assumes that some state has already been distributed. The reference bundle and model identity are shared per frozen study; the prompt, seed, tokenizer and contract configuration are shared per trial set; and correction positions and tokens are transferred per trajectory. The constructor must access the intended target environment during certificate construction. The recipient needs that matching target environment during replay. An auditor can verify hashes and decoded tokens without treating the correction manifest as a cryptographic proof of model execution. Correction tokens may reveal local information about the reference output and are not a privacy mechanism.

The sparse record stored the step and token identifier for each correction. We report three serialization boundaries: payload-only bytes, a self-contained JSON audit package, and a compact binary package that references a shared bundle. The historical fixed-width payload comparator used vocabulary size V, token identifiers of $\lceil \log_2(V)/8 \rceil$ bytes and step identifiers of $\lceil \log_2(n)/8 \rceil$ bytes. The new binary manifest uses versioned magic bytes and unsigned variable-length integers. Its referenced package header contains SHA-256 identifiers for the bundle, model and target environment. Ratios are always reported against the matching full-trace comparator, and the shared-bundle assumption is stated explicitly.

Outcome measures. The primary empirical outcome was correction density, defined as the number of stored correction pairs divided by generated tokens. Exact equality of corrected and reference token sequences was a protocol integrity gate: a trial that failed it was not treated as a successful replay, regardless of its correction density. Secondary outcomes were uncorrected exact replay, common exact prefix without corrections, payload-only and package-level record-size ratios, certificate bits per token, target-model passes, structural sentence completion, retained source mass, truncation component, full-logit quantization KL and total variation, shared-contract exactness, and reference-token coverage within target top-four, top-eight and top-16 envelopes. R048 quality materials are reported as preparation artifacts only. No human-quality endpoint is included.

Sentence completion was evaluated by a public punctuation-based predicate. It was applied only after token 64 and did not inspect the seed or correction manifest. This measure was used as a structural endpoint, not as a human-quality score.

Statistical analysis. The main continuous-metric analysis treated prompts as clusters and retained the three seed trajectories within each sampled prompt. Each trajectory contributed equally to the reported mean, so longer trajectories were not given greater weight. Means and two-sided 95% percentile intervals were calculated from 10,000 prompt-cluster bootstrap resamples using public bootstrap seed 20260720. English and Chinese strata each contained ten prompt clusters. For the top-*k* and precision-direction comparisons, conditions were paired by prompt and public seed. Bootstrap resampling preserved the paired prompt structure. Because top-*k* affects reference sampling, those comparisons were not paired token by token.

R053 was prespecified as a one-factor-at-a-time sensitivity analysis around $q = 0.5, T = 1.2, B = 16, k = 2$. It used all 20 prompts and public seed 0 for each variant. Correction density was aggregated as total corrections divided by total generated tokens. Paired differences resampled prompt identifiers 10,000 times with public seed 20260723. Parameter changes participate in reference sampling, so pairing was by prompt and seed rather than by token trajectory. All configured variants, including negative and interval-crossing results, remained in the analysis.

Exact counts are reported without null-hypothesis significance tests. For the main exact-replay result, all three trajectories succeeded in each of 20 prompt clusters, and a Wilson interval over prompt-level success was included as a conservative descriptive interval. No multiplicity-adjusted P values were calculated because the study reports prespecified effect estimates and uncertainty intervals rather than a family of significance claims. Trials were saved atomically and all configured trials were included. Analysis code and deterministic result signatures are provided with the repository.

1. Yuan, J. *et al.* Understanding and mitigating numerical sources of nondeterminism in LLM inference. *arXiv* 2506.09501 (2025).

2. Karvonen, A. *et al.* DiFR: inference verification despite nondeterminism. *arXiv* 2511.20621 (2025).

3. Wang, Y. *et al.* SparSamp: efficient provably secure steganography based on sparse sampling. In *34th USENIX Security Symposium* 6817-6835 (USENIX Association, 2025).

4. Yan, R. & Murawaki, Y. Efficient provably secure linguistic steganography via range coding. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics* 890-907 (Association for Computational Linguistics, 2026). https://doi.org/10.18653/v1/2026.acl-long.39.

5. Cao, W., Wang, Y. & Hu, D. Breaking the "provable security": detecting finite-precision artifacts in LLM-based steganography via low-probability vanishing. In *Findings of the Association for Computational Linguistics: ACL 2026* 20262-20274 (Association for Computational Linguistics, 2026). https://doi.org/10.18653/v1/2026.findings-acl.1013.

6. Qwen Team. Qwen2.5 technical report. *arXiv* 2412.15115 (2024).

7. Pang, K. & Bai, M. Provably secure steganography based on list decoding. *arXiv* 2604.21394 (2026).

8. Bar-Lev, D., Farnoud, F. & Gabrys, R. An additive approximation scheme for generating dyadic codings for the outputs of an LLM. *arXiv* 2605.05837 (2026).

9. Wang, Y. *et al.* ReTokSync: self-synchronizing tokenization disambiguation for generative linguistic steganography. *arXiv* 2604.25486 (2026).

Supplementary information Supplementary information accompanies this working paper.

Data availability The fixed bilingual prompt set, aggregate analysis files, Figure 1-4 source-data CSVs, generated PDF and 300-dpi PNG figures, R002 official-reproduction analysis, R050 integer-apportionment audit, R051 matched-baseline analysis, R052 unquantized-delta analysis, R053 bounded-sensitivity analysis, source hashes and deterministic result signatures are included in the project repository. Raw model outputs, private blinding keys and participant packages are excluded from version control and remain local experiment artifacts. The 1,200-trial R002 checkpoint and frozen reference-only bundle are transferred separately and verified by SHA-256 before analysis or external replay.

Code availability Code for probability contracts, replay certificates, checkpointed experiments, official-artifact compatibility analysis, external replay packaging, blind-study material generation and statistical analysis is available at `https://github.com/kdssss501/sparsamp-semantic` on the immutable Git tag `research-v0.41-final-integrity`, which includes the revised manuscript, R050-R053 evidence and Stage 4.5 integrity materials.

Ethics declaration The computational experiments used public model checkpoints, fixed prompts and locally generated outputs and did not involve human participants, identifiable personal data or animal subjects. The prepared R048 blinded-evaluation materials were not administered to participants and no human ratings are reported. Any future human evaluation will require prior institutional or supervisory determination of the applicable ethics, consent and data-management requirements.

AI-assisted work disclosure Generative AI tools were used during software development, experiment planning and manuscript drafting under author supervision. All executable changes were reviewed with automated tests or source inspection, quantitative claims were traced to saved experiment artifacts, and references were checked against official publication records or primary metadata sources. The authors remain responsible for the study design, code, analysis, interpretation and final text.

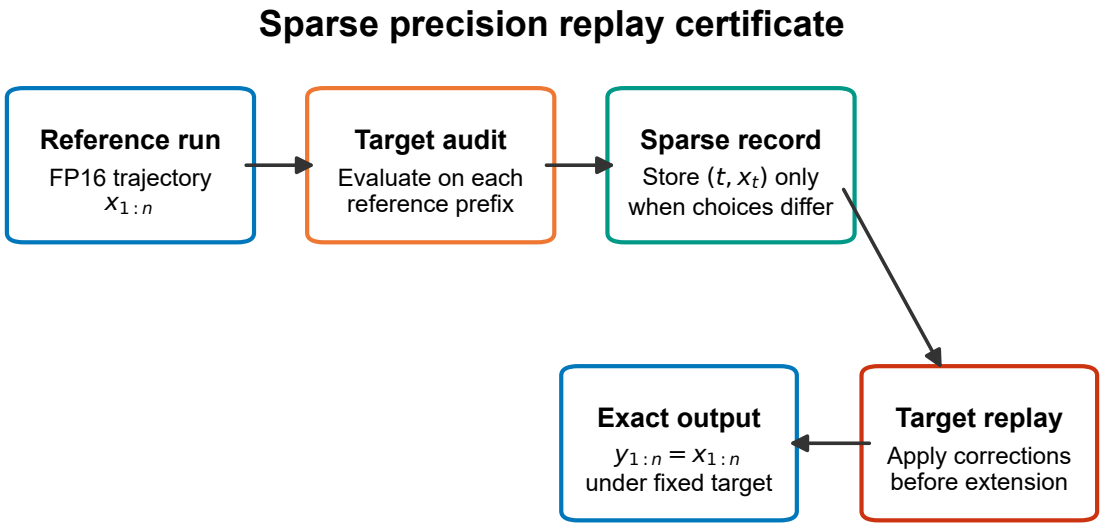

Figure 1 | Sparse precision replay certificate workflow. A reference-precision run generates a stochastic token trajectory from a public integer contract. The target precision evaluates the same contract on each reference prefix. Only target choices that differ from the reference token are stored. A fresh target run applies these corrections before extending its prefix, preventing autoregressive error propagation. The top-16 validation envelope is distinct from the top-two or top-four contract support.

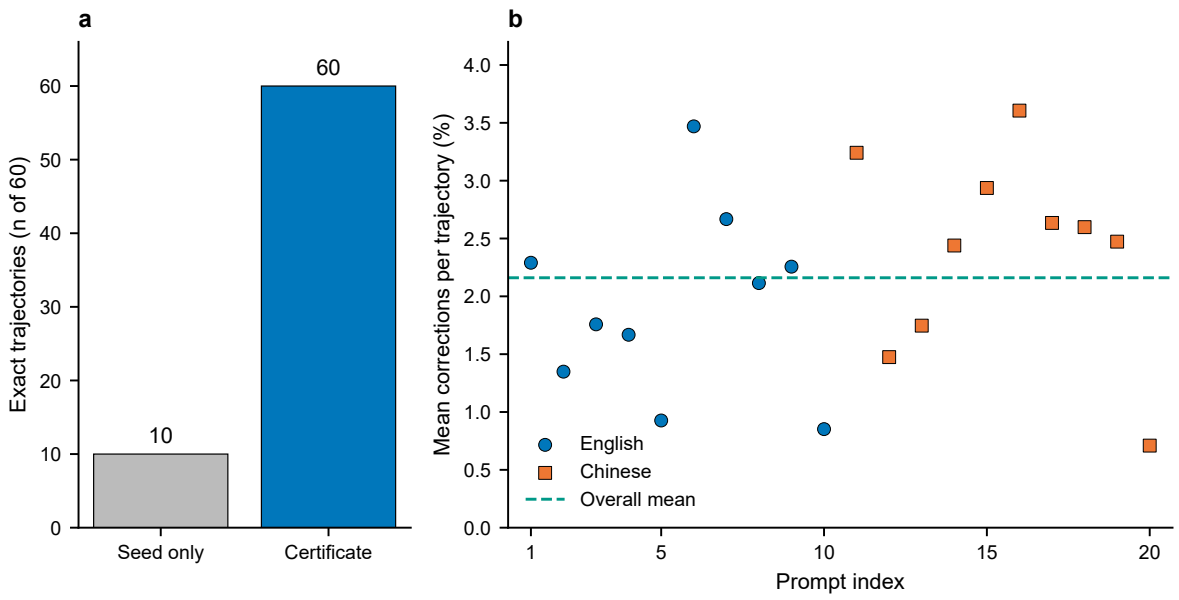

Figure 2 | Exact replay and sparse correction density across bilingual prompts. a, Corrected and uncorrected exact-replay counts for 60 FP16-to-BF16 trajectories. b, Mean correction rate for each of ten English and ten Chinese prompt clusters, with three seed trajectories retained within each prompt mean. The dashed line is the overall equal-trial mean.

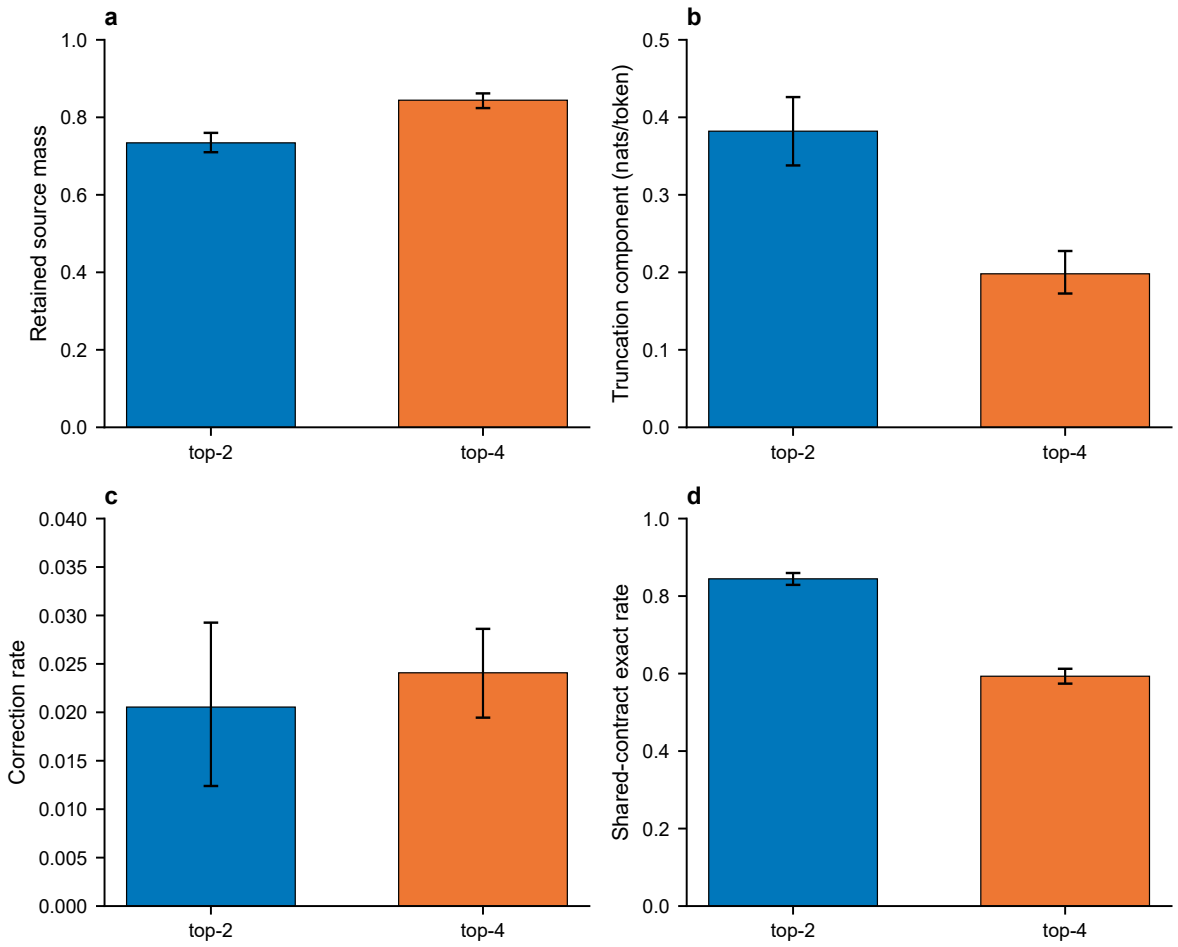

Figure 3 | Distribution-reliability trade-off with contract width. a, Retained source mass. b, Reverse-KL truncation component. c, Correction rate. d, Shared-contract exactness for top-two and top-four contracts on 20 matched prompts. Error bars are prompt-cluster bootstrap 95% intervals. The full-logit quantization term and paired differences are reported separately in the text and R046 analysis artifact.

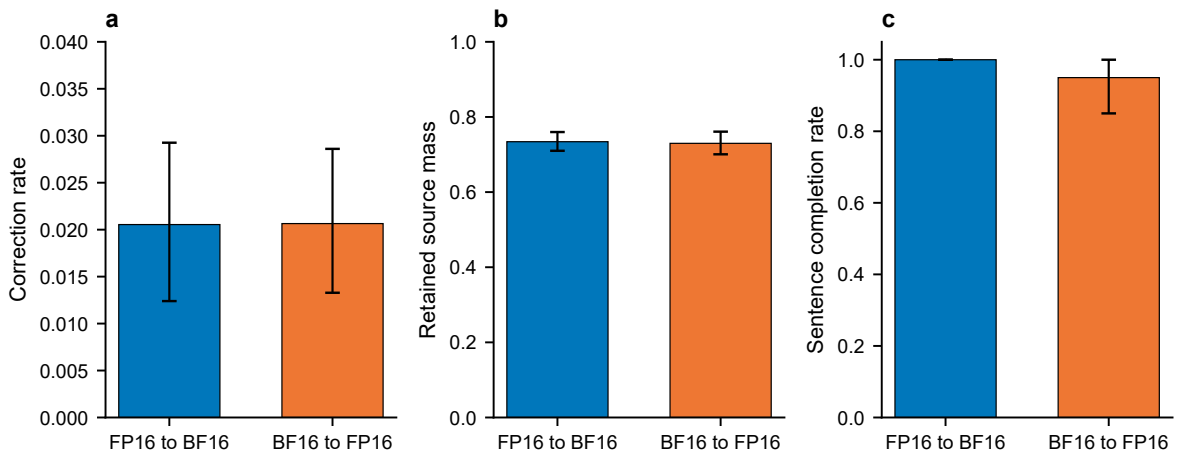

Figure 4 | Precision-direction stress test. Comparison of FP16-to-BF16 and BF16-to-FP16 top-two contracts on 20 matched prompts. Panels show a, correction density, b, retained source mass, and c, structural sentence completion. Both directions achieved 20 of 20 corrected exact replays, which are reported in Table 1. Error bars are prompt-cluster bootstrap 95% intervals.